

ET AI HACKATHON 2.0 — PHASE 2

Problem Statement 8: AI for Industrial Knowledge Intelligence — Unified Asset & Operations Brain

Industrial Knowledge Copilot

One AI brain for every manual, log, and lesson your plant has ever written down.

Team: JUMechParth

Submitted by: Parth Pariwandh
Jadavpur University, Kolkata

Submission Document — Detailed Technical Proposal
[Insert submission date]

Table of Contents

Executive Summary..... 3

Problem Statement.....3

Why This Problem Matters.....4

Proposed Solution..... 4

Core Features..... 5

System Architecture.....5

Data Flow.....6

Tech Stack..... 6

Model / Pipeline Design.....7

User Workflow..... 8

Demo Scenario.....8

Expected Impact.....9

Evaluation Metrics..... 9

Limitations and Assumptions.....10

Future Scope..... 10

Conclusion.....11

References / Source Placeholders..... 11

Executive Summary

Asset-intensive Indian industries lose an estimated 35% of working hours to searching for information that already exists somewhere in the organisation — scattered across P&IDs, maintenance work orders, SOPs, inspection reports, and OEM manuals held in 7 to 12 disconnected systems per plant. This fragmentation is not a filing problem; it is a safety, quality, and continuity problem that worsens every year as roughly a quarter of India's experienced industrial engineers approach retirement, taking undocumented operational knowledge with them.

Industrial Knowledge Copilot is a document-centric AI platform that turns this scattered paper trail into a single, queryable knowledge layer. It ingests heterogeneous industrial documents, extracts the entities and relationships that matter (equipment tags, parameters, regulatory references, personnel, dates), and exposes that knowledge through a retrieval-augmented conversational interface that answers operational questions with direct citations back to source pages. A lightweight knowledge graph links related documents and entities, while a compliance and maintenance insight layer surfaces regulatory gaps and recurring failure patterns automatically.

The platform is deliberately scoped for rapid, credible prototyping: it relies entirely on open-source parsing, embedding, and retrieval components, requires no live sensor or SCADA integration, and can be demonstrated end-to-end using a small, realistic corpus of sample industrial documents. This document presents the complete technical proposal — problem framing, architecture, pipeline design, demo plan, evaluation approach, and roadmap — for the Industrial Knowledge Copilot submission to ET AI Hackathon 2.0, Phase 2.

Problem Statement

Official Challenge Statement

Build an AI-powered Industrial Knowledge Intelligence platform that ingests heterogeneous documents — engineering drawings, maintenance records, safety procedures, inspection reports, operating instructions, project files — across structured and unstructured formats, and makes their collective intelligence queryable, actionable, and continuously updated at the point of need, across any device or function.

Context

A 2024 McKinsey global survey found that professionals in asset-intensive industries spend roughly a third of their working hours searching for information or recreating documents that already exist. A NASSCOM-EY study of Indian manufacturing and energy companies found that a typical large plant operates across 7 to 12 disconnected document systems, and BIS Research estimates this fragmentation contributes to 18-22% of unplanned downtime events in Indian heavy industry, since maintenance teams frequently make decisions without complete equipment history or failure-pattern context.

Layered on top of this is the knowledge cliff: an estimated 25% of India's experienced industrial engineers and operators will retire within the next decade, taking with them operational knowledge that was never written down in a retrievable form. Once that knowledge leaves with a person, it cannot be reconstructed from a filing cabinet.

Why This Problem Matters

- **Safety:** incomplete equipment history and scattered inspection records mean hazard signals that exist on paper never reach the people who need them in time.
- **Quality:** regulatory and procedural compliance checks performed manually against Factory Act, OISD, PESO, and internal QMS standards are slow, inconsistent, and easy to miss under audit pressure.
- **Operational efficiency:** 18-22% of unplanned downtime is linked to fragmented information access — a direct, quantifiable cost that a working knowledge layer can attack immediately.
- **Institutional continuity:** tacit knowledge held by retiring engineers is the hardest asset to replace; capturing it in a queryable form is the only scalable mitigation.

Because this is a knowledge-access problem rather than a sensor-integration or computer-vision problem, it is one of the few problem statements in this hackathon where a small team can build something genuinely useful — not just a demo — within the time available.

Proposed Solution

Industrial Knowledge Copilot is an AI-powered document intelligence platform purpose-built for industrial teams. It ingests PDFs, scanned manuals, maintenance logs, SOPs, inspection reports, and safety documents, and turns them into a single source of truth that any engineer, technician, or auditor can query in plain language.

The platform is organised around six capabilities that map directly onto the official challenge statement's suggested build areas: universal ingestion with lightweight knowledge-graph linking, a citation-backed expert copilot, maintenance and failure-pattern insight extraction, regulatory compliance gap detection, and a clean web dashboard that ties all of it together. Each capability is implemented as a thin, well-tested layer rather than a sprawling system — favouring a strong document-centric core over a complex multi-system integration that a small team could not realistically deliver and demonstrate within a hackathon timeline.

Problem-to-Solution Mapping

Challenge Statement Build Area	Industrial Knowledge Copilot Feature
Universal Document Ingestion & Knowledge Graph Agent	Ingestion pipeline + lightweight entity graph (NetworkX + spaCy NER)
Expert Knowledge Copilot	RAG-based Q&A with page-level citations and confidence scores

Challenge Statement Build Area	Industrial Knowledge Copilot Feature
Maintenance Intelligence & RCA Agent	Maintenance insight extraction module surfacing failure-pattern summaries
Quality & Regulatory Compliance Intelligence	Compliance gap detector mapping procedures against a regulatory reference corpus
Lessons Learned & Failure Intelligence Engine	Roadmap item — see Future Scope

Table 1: Mapping of official challenge build areas to platform features.

Core Features

1. Universal Document Ingestion — accepts PDFs, scanned images, and common office formats; auto-detects digital-text vs. scanned-image pages and routes each accordingly.
2. OCR / Parsing Layer — extracts clean text and layout structure from scanned and native-PDF documents, preserving page and section references for later citation.
3. RAG-Based Expert Copilot — answers natural-language questions across the full document corpus with inline citations back to the exact source page.
4. Lightweight Knowledge Graph — links equipment tags, parameters, personnel, and regulatory references across documents so related records surface together.
5. Compliance Gap Detector — checks ingested SOPs and inspection records against a reference set of regulatory clauses and flags unaddressed or contradicted requirements.
6. Maintenance & Failure Insight Extraction — summarises recurring failure descriptions and maintenance actions across logs to highlight patterns a single reviewer would miss.
7. Clean Web Dashboard — single interface for upload, chat-style Q&A, document browsing, and a compliance/maintenance insight feed.

System Architecture

The architecture is organised into five layers — ingestion, processing, knowledge storage, retrieval and reasoning, and presentation — each independently testable and individually simple. A full diagram-ready component breakdown with labelled boxes and arrows is provided as a separate deliverable (Architecture Diagram); the layers are summarised below.

Layer	Components	Responsibility
Ingestion	Upload API, format router	Accepts files, classifies as native-text or scanned
Processing	OCR engine, parser, chunker, NER	Extracts text/layout, splits into citable chunks, tags entities
Knowledge Storage	Vector store, lightweight graph	Persists embeddings, entity

Layer	Components	Responsibility
	store, metadata DB	relationships, and document metadata
Retrieval & Reasoning	Retriever, re-ranker, LLM answer generator, compliance/insight engine	Finds relevant chunks, generates cited answers, flags gaps and patterns
Presentation	Web dashboard, chat UI, document viewer	Surfaces all of the above to the end user

Table 2: System architecture layers and responsibilities.

Data Flow

1. A user uploads one or more documents (PDF, scanned image, or office file) through the web dashboard.
2. The format router inspects each page and sends scanned pages to OCR and native-text pages directly to the parser.
3. Extracted text is cleaned, segmented into overlapping chunks, and each chunk is tagged with its source document, page number, and section heading for later citation.
4. Named-entity recognition extracts equipment tags, parameters, dates, personnel, and regulatory references from each chunk; these populate the lightweight knowledge graph.
5. Each chunk is embedded and stored in the vector store alongside its metadata.
6. When a user asks a question, the retriever fetches the most relevant chunks, the re-ranker orders them by relevance, and the LLM generates an answer that cites the originating document and page.
7. In parallel, the compliance engine checks newly ingested SOPs and inspection records against the regulatory reference corpus, and the insight engine scans maintenance logs for recurring failure language; both push flagged items to the dashboard feed.

Tech Stack

Layer	Tool / Library	Why
Frontend	React + Tailwind (or Streamlit for fastest build)	Fast to build, familiar, sufficient for a focused dashboard
Backend API	FastAPI (Python)	Lightweight, async-friendly, quick to wire to ML components
OCR	Tesseract OCR / EasyOCR	Mature, open-source, handles scanned industrial forms reasonably well
PDF / Layout	PyMuPDF, pdfplumber,	Reliable text and layout extraction

Layer	Tool / Library	Why
Parsing	unstructured.io	with page-level coordinates
NER / Entity Extraction	spaCy (custom patterns for equipment tags & regulation refs)	Open-source, fast to extend with domain rules
Embeddings	Open-source sentence-transformer (e.g., bge-small) or hosted embeddings API	Cost-effective; swappable without re-architecting
Vector Store	ChromaDB / FAISS	Local, zero-infrastructure, ideal for a demo-scale corpus
Knowledge Graph	NetworkX (in-memory graph) + SQLite for persistence	Avoids standing up a graph database for a prototype-scale corpus
LLM (answer generation)	Claude API	Strong citation-following and instruction adherence for grounded answers
Deployment	Docker Compose, single VM or laptop demo	Reproducible, judge-runnable without cloud dependencies

Table 3: Recommended technology stack, with rationale for each choice.

Model / Pipeline Design

Chunking & Citation Strategy

Documents are split into overlapping chunks of roughly 250-400 tokens, with each chunk carrying a stable identifier composed of document ID, page number, and chunk offset. This identifier travels with the chunk through embedding and retrieval, so every sentence the LLM produces can be traced back to an exact page — the core requirement for a copilot that engineers will actually trust on a safety-critical question.

Retrieval & Answer Generation

At query time, the top-k chunks by embedding similarity are retrieved, re-ranked using a lightweight cross-encoder (or, at minimum, recency and document-type heuristics if time is short), and passed to the LLM with an explicit instruction to answer only from the provided chunks and to cite the document and page for every claim. Answers below a confidence threshold are flagged as "low confidence — verify against source" rather than presented as fact.

Entity Extraction & Knowledge Graph

A small set of domain entity types is targeted deliberately rather than attempting a general-purpose ontology: equipment tag, process parameter, regulatory reference, personnel/role, and date. Equipment tags and regulatory references are extracted with rule-based patterns (regular expressions tuned to common tag formats) backed by spaCy's statistical NER for personnel and

dates. Extracted entities become graph nodes; co-occurrence within the same chunk becomes an edge. This produces a usable graph without requiring manual ontology engineering.

Compliance Gap Detection

A small reference corpus of regulatory clauses (Factory Act, OISD, or internal QMS extracts relevant to the demo scenario) is ingested the same way as any other document. For each SOP or inspection record, the system retrieves the regulatory clauses most semantically similar to its content and asks the LLM to assess, clause by clause, whether the procedure addresses, partially addresses, or omits the requirement — producing a short, auditable gap list rather than a black-box score.

Maintenance & Failure Insight Extraction

Maintenance log entries are clustered by embedding similarity to surface recurring failure descriptions across equipment and time. The LLM is then prompted to summarise each cluster into a short pattern statement (for example, a recurring seal failure on a specific pump model), which is presented to the user as a candidate root-cause lead rather than a final diagnosis.

User Workflow

1. Upload — an engineer or safety officer drags a batch of documents into the dashboard; the system shows ingestion progress per file.
2. Browse — uploaded documents appear in a searchable library, grouped by type (manual, SOP, maintenance log, inspection report).
3. Ask — the user opens the copilot chat and asks an operational question in plain language.
4. Verify — the answer appears with citations; clicking a citation opens the exact source page alongside the answer.
5. Review insights — a side panel surfaces newly flagged compliance gaps and maintenance patterns relevant to the documents just ingested.
6. Export — the user can export a question-and-answer thread or a compliance gap list as a shareable summary for an audit or handover.

Demo Scenario

The live demo uses a small, realistic corpus assembled specifically for this submission: one OEM equipment manual, two maintenance log excerpts, one SOP, and one inspection report (see Suggested Datasets / Sample Documents for sourcing guidance). The walkthrough below is designed to fit a 3-4 minute demo video.

1. Upload all five sample documents and show the ingestion progress bar completing in under a minute.

2. Ask a direct factual question (for example, the recommended lubrication interval for a named pump) and show the cited answer with the source page opening alongside it.
3. Ask a cross-document question that requires combining the manual and a maintenance log (for example, whether a logged failure matches a known failure mode in the manual) and show the answer drawing on both sources.
4. Open the compliance panel and show a flagged gap where the SOP does not fully address a clause from the reference regulatory corpus.
5. Open the maintenance insight feed and show a surfaced pattern across the two maintenance log excerpts.

[Insert Screenshot 1: Upload & ingestion progress view]

[Insert Screenshot 2: Copilot chat with cited answer and source page panel]

[Insert Screenshot 3: Compliance gap feed]

[Insert Screenshot 4: Maintenance insight feed]

Expected Impact

- Time-to-answer for routine operational questions drops from minutes of manual document searching to seconds, with full source traceability.
- Compliance reviews that currently rely on a person manually cross-referencing procedures against regulations gain a continuously available first-pass check, reducing the chance that a gap surfaces only during an external audit.
- Recurring failure patterns that are currently visible only to whoever happens to read multiple logs become visible automatically, supporting earlier maintenance intervention.
- Knowledge captured from retiring engineers' documentation is preserved in a queryable form rather than existing only in their memory or in a folder nobody else reads.

These are presented as directional, prototype-stage impact hypotheses rather than measured production results; validating them at plant scale is explicitly listed under Future Scope.

Evaluation Metrics

These metrics align directly with the official evaluation focus areas published for this problem statement, scoped to what a prototype-stage corpus can credibly demonstrate.

Metric	What It Measures	How It Is Assessed in the Demo
Entity extraction accuracy	Correctness of equipment tags, parameters, dates, and regulatory references pulled from sample documents	Manual spot-check against a small hand-labelled subset of the demo corpus

Metric	What It Measures	How It Is Assessed in the Demo
Query answer quality	Whether copilot answers are correct and properly grounded in cited sources	A fixed benchmark set of domain questions, scored for correctness and citation validity
Knowledge graph linkage completeness	Whether related entities across documents are correctly connected	Manual review of graph edges for the demo corpus
Time-to-answer vs. traditional search	Speed advantage of the copilot over manual document search	Timed comparison on the same benchmark questions
Compliance gap detection accuracy	Whether flagged gaps match gaps identified by manual review	Comparison against a manually reviewed gap list for the demo SOP

Table 4: Evaluation metrics mapped to the official judging focus areas.

Limitations and Assumptions

- The prototype is built and validated against a small, curated document corpus, not an enterprise-scale archive; ingestion throughput and retrieval quality at scale are not demonstrated.
- OCR accuracy on poor-quality or handwritten scans is a known weak point of open-source OCR engines and may require manual correction in real deployments.
- The knowledge graph is intentionally lightweight (entity co-occurrence, not a full industrial ontology) and should be read as a relationship surface, not a verified engineering model.
- The compliance reference corpus used in the demo is a representative sample of regulatory clauses, not an exhaustive or legally authoritative compliance database.
- Documents are assumed to be in English and either digitally native or legibly scanned; multi-language and severely degraded scans are out of scope for this prototype.

Future Scope

- Lessons Learned & Failure Intelligence Engine — extend pattern detection across incident reports, near-miss records, and external industry databases, as outlined in the official challenge statement's fifth build area.
- Mobile-first field interface so technicians can query the copilot directly from the plant floor, not only from a desktop dashboard.
- Multi-language support for regional-language SOPs and inspection records.
- Integration with live CMMS/SCADA feeds to connect static document knowledge with real-time equipment state.

- Domain fine-tuning of embeddings on a larger, anonymised industrial corpus to improve retrieval precision beyond what general-purpose embeddings provide.
- Expansion of the regulatory reference corpus toward full coverage of Factory Act, OISD, and PESO requirements relevant to a given plant type.

Conclusion

Industrial Knowledge Copilot addresses Problem Statement 8 with a document-centric design that is honest about what a small team can build and demonstrate convincingly within a hackathon timeline, while still covering every build area named in the official challenge statement: universal ingestion with lightweight knowledge-graph linking, a citation-backed expert copilot, maintenance insight extraction, and regulatory compliance gap detection. The result is a platform that turns a plant's scattered manuals, logs, and procedures into a single, trustworthy source of answers — and a credible first step toward preserving the operational knowledge that India's heavy industry is at real risk of losing.

References / Source Placeholders

- McKinsey & Company (2024). Global survey on time spent searching for information in asset-intensive industries. [Insert full citation]
- NASSCOM-EY (2024). Study on document system fragmentation in Indian manufacturing and energy companies. [Insert full citation]
- BIS Research (2024). Estimate of unplanned downtime attributable to information fragmentation in Indian heavy industry. [Insert full citation]
- ET AI Hackathon 2.0 — Phase 2, Official Problem Statements document, Problem 8. [Insert organiser reference / link]
- [Insert citation for any OEM manual or regulatory text used as a sample document, with permission/source noted]

[Insert any additional source links used while assembling the sample document corpus]